

OOP MID PAPER:

MUHAMMAD ALI ABID

25021598-010

DEFINITIONS

Q-1 . Define **class and object** in Java and explain how objects access **static and non-static members** with a simple example.

Q-2. Explain **method overloading** and describe how Java decides which method to call, including the role of **objects and static methods**.

Q-3. Define **static and non-static members** and explain why a **static method cannot directly access instance variables**.

Q-4. Explain **access modifiers (private, public)** and how they support **encapsulation using methods and objects**.

Q-5. Define **encapsulation and abstraction** and explain how they are achieved using **classes, objects, methods, and access control**.

Q-6. what are **conditional statements** and also define types of conditional statements with examples.

Q-7. Define **loop** and types of loops with their important **characteristics**.

ERROR FINDING:

```
public class Test1 {  
    1-    public static void main(String[]  
args) {        int marks = 85  
  
        if(marks >= 80)
```

```

        System.out.println("A Grade")
else    if(marks    >=    60)
System.out.println("B Grade");
    else
        System.out.println("Fail");
    }
}

```

```

2-    public class Test2 {    public static
void main(String[] args) {    int day = 2;

    switch(day) {
case 1:
        System.out.println("Monday");
    case 2:
        System.out.println("Tuesday");
        break;
case 3
        System.out.println("Wednesday");
        break;
default:
        System.out.println("Invalid");
    }
}
}

```

```

3-    public class Test3 {    public static void main(String[] args) {
for(int i = 1; i <= 5; i++)
    {

```

```
        System.out.println(i);  
    }
```

```
        while(i < 10) {  
            System.out.println(i);        i++;  
        }  
    }  
}
```

4- **public class Test4 {**

```
    int add(int a, int b) {  
        return a + b;  
    }
```

```
    int add(int x, int y) {  
        return x + y;  
    }
```

```
    public static void main(String[] args) {  
        Test4 obj = new Test4();  
        System.out.println(add(2,3));  
    }  
}
```

5- **public class Test5 {**

```

static int count = 0;

void increment() {
count++;
}

public static void main(String[] args) {
increment();
    System.out.println(count);
}
}

```

OUTPUT FINDING:

```

1- public class Test1 { public
static void main(String[] args) { int
sum = 0;

    for(int i = 1; i <= 5; i++) {
if(i % 2 == 0) sum
+= i; else
sum -= i;
    }

    System.out.println(sum);
}
}

```

```

2- public class Test2 { public
static void main(String[] args) {
for(int i = 1; i <= 3; i++) {      for(int
j = 1; j <= i; j++) {
        System.out.print(i + j + " ");
    }
    System.out.println();
}
}
}

```

```

3- public class Test3 { public
static void main(String[] args) {      int
x = 2;

```

```

        switch(x) {      case 1:
System.out.print("A      ");
case      2:
System.out.print("B      ");
case      3:
System.out.print("C      ");
break;      default:
        System.out.print("D ");
    }
}
}
}

```

```

4- public class Test4 {

```

```

static int count = 0;

void increment() {
count++;
}

public static void main(String[] args) {
Test4 obj1 = new Test4();    Test4
obj2 = new Test4();

    obj1.increment();
    obj2.increment();
obj1.increment();

    System.out.println(count);
}
}

```

```

5- public class Test5 {

    int add(int a, int b) {
return a + b;
    }

    int add(int a, int b, int c) {
return a + b * c;
    }

```

```

public static void main(String[] args) {
    Test5 obj = new Test5();

    int result1 = obj.add(2, 3);
    int result2 = obj.add(2, 3, 4);

    System.out.println(result1 + " " + result2);
}
}

```

SHORT PROGRAMS:

1 - Scholarship System

Statement:

Write a Java program that takes: •

Student marks

- Attendance percentage
- Family income

Apply the following logic:

- If attendance < 70% → Not eligible
- Otherwise:
 - If marks $\geq$ 85:

□ If income < 30000 → Full scholarship

□ Else → Half

scholarship ○ Else if marks $\geq$ 70:

□ If income < 30000 → Half scholarship

□ Else → No

scholarship ○ Else → No scholarship

2- Banking Menu System

Statement:

Create a menu-driven program using switch:

1. Deposit
2. Withdraw
3. Check Balance

Rules:

- Initial balance = 5000
- For Withdraw:
 - If amount > balance → “Insufficient Balance”
 - If amount > 20000 → “Limit Exceeded”
- For Deposit:
 - If amount > 50000 → Ask for confirmation

3- Number Analyzer**Statement:**

Write a program that:

- Takes 10 numbers using a loop
- Count how many are:
 - Even
 - Odd
 - Positive
 - Negative
- Also find:
 - Largest number
 - Second largest number

4- Pattern + Prime**Check****Statement:**

Write a program that:

- Prints the following pattern using nested loops:

3 5
7 11 13
17 19 23 29

- Each number in the pattern must be **prime numbers only**
- Generate primes dynamically (not hardcoded)

5- Create a program that:

- Uses a loop to repeatedly show menu:
 1. Add 2 numbers
 2. Add 3 numbers
 3. Multiply
 4. Check even/odd
- Use switch for menu
- Use **method overloading**:
 - add(int a, int b)
 - add(int a, int b, int c)
- Create separate methods for: ○ Multiply ○ Even/Odd check
- Program runs until user chooses Exit

LONG PROGRAMS:

1. Smart University Management System

Statement:

Design a Java program for a university system using **OOP concepts**.

Requirements:

- Create a class Student with:
 - Attributes: name, marks, attendance, feePaid
- Create:
 - **Non-static method** → to display student details
 - **Static method** → to calculate grade

Program Logic:

- Use a **loop** to enter data for multiple students

- For each student:
 - Use **nested if**:
 - If attendance < 75% → Not allowed in exam
 - Else:
 - If marks ≥ 85 → Grade A
 - 70–84 → Grade B
 - 50–69 → Grade C
 - < 50 → Fail
- Use **method overloading**:
 - calculateFee(int creditHours)
 - calculateFee(int creditHours, boolean scholarship)
- Create a **menu using switch**:
 1. Add Student
 2. Display All Students
 3. Search Student
 4. Exit
- Add logic:
 - If fee not paid → restrict result display
 - Use **objects** to store student records

2. Advanced Banking System with Security

Statement:

Develop a Java program using **class BankAccount**.

Attributes:

- accountHolder, balance, pin

Methods:

- **Non-static methods**:
 - deposit() ◦ withdraw() ◦ checkBalance()
- **Static method**:

- bankPolicy() → displays rules

Program Logic:

- Use a **loop** to continuously show menu:
 1. Deposit
 2. Withdraw
 3. Balance Inquiry
 4. Exit
 - Use **switch** for menu
 - Apply **nested if**:
 - For withdraw:
 - If pin is correct:
 - If balance sufficient:
 - If amount ≤ 25000 → allow
 - Else → require manager approval
 - Else → block transaction
 - Use **method overloading**:
 - deposit(int amount) ○ deposit(int amount, String source)
 - Add **loop-based login system**:
 - User gets 3 attempts for correct PIN
 - Use **objects** to manage multiple accounts
-

3. Smart Shopping & Billing System

Statement:

Create a Java-based shopping system using a class Product.

Attributes:

- productName, price, quantity

Methods:

- **Non-static methods:**
 - calculateTotal() ○ displayBill()

- **Static method:**

- showDiscountPolicy()

Program Logic:

- Use a **loop** to allow multiple product entries
- Use **nested loop** to:
 - Display bill in tabular form
- Apply **nested if** for discount:
 - If total > 5000:
 - If customer is member
→ 20% discount
 - Else → 10% discount ◦
 - Else if total > 2000 → 5% discount ◦
 - Else → no discount
- Use **method overloading**:
 - calculateTotal(int price, int quantity) ◦ calculateTotal(int price, int quantity, int discount)
- Use **switch menu**:
 1. Add Product
 2. Generate Bill
 3. Show Discount Policy
 4. Exit
- Add logic:
 - If quantity = 0 → reject product ◦
 - If price is negative → show error